

PROJECT DESIGN PHASE

SOLUTION ARCHITECTURE

Date	30 September 2026
Team ID	SWTID-2026-3024
Project Name	AI-Augmented Backend Development AI BlogNest API
Maximum Marks	4 Marks

Solution Architecture

The AI BlogNest system follows a modular and layered backend architecture designed to provide secure blog content management and AI-assisted content generation. The architecture connects the client application with the backend API, authentication and authorization mechanisms, database services, and Artificial Intelligence services.

The client sends requests to the Node.js and Express.js backend through RESTful APIs. The incoming requests are processed through appropriate middleware for authentication, authorization, validation, and security checks. After successful verification, the request is forwarded to the relevant controller.

The controllers handle the application logic and communicate with the required services and database models. Mongoose is used to interact with the MongoDB database for storing and retrieving users, blog posts, comments, and related information.

For AI-related operations, such as blog content generation and content summarization, the backend communicates with the configured AI service. The generated AI response is processed by the backend and returned to the client in a structured JSON format.

Architecture Components

1. Client / API Client

The client sends HTTP requests to the backend API and receives the processed responses. Postman can be used as an API testing client during development.

2. Express.js Server

Express.js handles API routing, request processing, middleware execution, and communication between different backend components.

3. Middleware Layer

The middleware layer performs authentication, authorization, validation, sanitization, and other request-processing activities.

4. Controller Layer

Controllers process incoming requests and coordinate the required operations between the client, services, and database models.

5. Service / AI Layer

The service layer contains reusable business logic. AI-related operations are handled through the integrated AI service for generating and summarizing blog content.

6. MongoDB Database

MongoDB stores application data such as user information, blog posts, comments, and other content-related records.

7. Mongoose

Mongoose provides schema-based interaction between the Node.js application and MongoDB and supports database validation and operations.

8. Security Layer

JWT authentication, bcrypt.js password hashing, role-based authorization, input validation, and sanitization are used to improve the security of the application.

Architecture Data Flow

Client → Express.js Server → Middleware → Controller → Service / Model → MongoDB / AI Service → Response

The architecture separates the major responsibilities of the system into different components. This modular structure makes AI BlogNest easier to develop, test, maintain, and extend with additional features in the future.

Figure 1: AI BlogNest Solution Architecture

